PART 12 — DEPLOYMENT & DEVOPS SRE RUNBOOK

Hosting Infrastructure: AWS ECS / Aurora PostgreSQL (`eu-west-2` London)

Containerization: Multi-Stage Docker (`Dockerfile.api`, `Dockerfile.web`)

CI/CD Pipeline: GitHub Actions (`.github/workflows/ci.yml`)

Database Tooling: Prisma ORM Migrations (`prisma migrate deploy`)

1. Environment Architecture (Non-Technical Explanation)

Software requires rigorous testing before it touches real financial records. blynt uses three isolated environments:

1. **Local Development (The Workshop):** Runs on developers' laptops using embedded PostgreSQL. Engineers build and test new features here without any risk to real customer data.
2. **Staging / UAT (The Dress Rehearsal):** A production-identical clone using synthetic test company data. Accountants test new workflows here to verify calculations and UX flows before releases.
3. **Production (The Live Bank):** The live system in AWS London (`eu-west-2`). Zero-downtime deployments ensure businesses can invoice customers 24/7 without service interruption.

2. Multi-Stage Docker Containerization

2.1 Backend Container (`Dockerfile.api`)

`dockerfile

FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./

COPY apps/api/package*.json ./apps/api/

RUN npm ci

COPY apps/api ./apps/api

RUN npx prisma generate --schema=./apps/api/prisma/schema.prisma

RUN npm run build --workspace=apps/api

FROM node:20-alpine AS runner

WORKDIR /app

ENV NODE_ENV=production

COPY --from=builder /app/package*.json ./

COPY --from=builder /app/apps/api/package*.json ./apps/api/

RUN npm ci --omit=dev

COPY --from=builder /app/apps/api/dist ./apps/api/dist

COPY --from=builder /app/apps/api/prisma ./apps/api/prisma

COPY --from=builder /app/node_modules/.prisma ./node_modules/.prisma

USER node

EXPOSE 4000

CMD ["node", "apps/api/dist/src/main.js"]

2.2 Frontend Container (`Dockerfile.web`)

- Uses Next.js 15 Standalone mode (`output: 'standalone'`).
- Copies minimal server bundle and `.next/static` assets.
- Final container image size is **< 120MB**, ensuring rapid auto-scaling during traffic spikes.

3. Zero-Downtime Database Migration Runbook

Database schema updates in a 24/7 accounting platform must never lock tables or break active transactions.

Step 1: Pre-Deployment Schema Verification

`bash

npx prisma migrate deploy --schema=./apps/api/prisma/schema.prisma

Step 2: Backward-Compatible Schema Changes (The Expand-Contract Pattern)

4. **Never rename a column directly:** Renaming `name` to `legal_name` breaks old running containers before new ones start.
5. **Phase 1 (Expand):** Add the new column as nullable. Deploy new API version that writes to both columns.
6. **Phase 2 (Migrate):** Run a background script to backfill data from old column to new column.
7. **Phase 3 (Contract):** Drop the old column in a subsequent release once all containers are upgraded.

4. Emergency Rollback Procedures

If an alert triggers during deployment (e.g., error rate > 0.5% or health check timeout on `/api/v1/health`):

8. **Automated Traffic Abort:** AWS ECS automatically redirects traffic back to the previous stable task definition.
9. **Database Rollback:** If the release included a migration that must be reverted, execute the reverse SQL script located in `docs/aws/rollback-runbook.md`.
10. **Incident Post-Mortem:** Post-deployment review logged within 24 hours in the team risk register.